

QRing Android SDK External Integration Guide

This guide is generated from the `SDKSample` project and is intended for external Android integrators.

1. Requirements

1.1 Runtime Requirements

- Device: QC QRing wearable devices.
- Android: the legacy SDK document requires Android 5.0+ and Bluetooth 4.0+; the current Sample project uses `minSdkVersion 26`.
- SDK file: `app/libs/qring_sdk_1.0.0.xx.aar`.
- Sample entry point: `HomeActivity`.

1.2 Gradle Setup

Configure the local AAR in your app module:

```
android {
    repositories {
        flatDir {
            dirs 'libs'
        }
    }
}

dependencies {
    implementation(name: 'qring_sdk_1.0.0.xx', ext: 'aar')
}
```

The recording Opus decoder is packaged with the SDK AAR, so host applications do not need to add the `j1_audio_decode` AAR separately. The Sample also uses ViewBinding, Kotlin, EventBus, XXPermissions, MMKV, XLog, and AspectJ. External projects only need to keep the dependencies they actually use.

2. Permissions

2.1 Bluetooth and Location

```
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />

<!-- Android 12+ -->
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
<uses-permission android:name="android.permission.BLUETOOTH_SCAN" />
```

```
<uses-permission android:name="android.permission.BLUETOOTH_ADVERTISE" />
```

Runtime Bluetooth and location permissions are required before scanning BLE devices. See:

- `utils/PermissionUtil.kt`
- `DeviceBindActivity`
- `HomeActivity`

3. SDK Initialization

Initialize the SDK in `Application.onCreate()`:

```
class MyApplication : Application() {  
    override fun onCreate() {  
        super.onCreate()  
        BleOperateManager.getInstance(this)  
        BleOperateManager.getInstance().init()  
    }  
}
```

3.1 Required SDK In-App Broadcast Receiver

`MyBluetoothReceiver` is important. It extends the SDK `QCBluetoothCallbackCloneReceiver` and receives BLE callbacks that the SDK sends as app-scoped broadcasts. External apps must register it dynamically in `Application.onCreate()`, otherwise connection, service-discovery, characteristic read/change, and descriptor callbacks will not be received.

Sample registration:

```
class MyApplication : Application() {  
    override fun onCreate() {  
        super.onCreate()  
  
        val sdkReceiver = MyBluetoothReceiver()  
        ContextCompat.registerReceiver(  
            applicationContext,  
            sdkReceiver,  
            BleAction.getIntentFilter(),  
            ContextCompat.RECEIVER_NOT_EXPORTED  
        )  
  
        BleOperateManager.getInstance(this)  
        BleOperateManager.getInstance().init()  
    }  
}
```

`BleAction.getIntentFilter()` already includes SDK connection and GATT actions such as:

- `BLE_GATT_CONNECTED` / `BLE_GATT_DISCONNECTED`
- `BLE_SERVICE_DISCOVERED`
- `BLE_CHARACTERISTIC_READ`
- `BLE_CHARACTERISTIC_CHANGED`

- `BLE_DESCRIPTOR_WRITE`

`MyBluetoothReceiver` should handle at least these callbacks:

- `connectStatue(device, connected)`: update connection state; notify the UI or business layer when the device disconnects.
- `onServiceDiscovered()`: call `LargeDataHandler.getInstance().initEnable()` after service discovery, then notify the business layer that commands can be sent. After this callback, first request `SetTimeReq` and `DeviceSupportReq.getReadInstance()` serially; after the support-capability responses are returned and cached, sync or query health data.
- `onCharacteristicRead(uuid, data)`: read firmware and hardware version characteristics. The Sample caches values from `Constants.CHAR_FIRMWARE_REVISION` and `Constants.CHAR_HW_REVISION`.
- `onCharacteristicChange(address, uuid, data)`: handle device notify data if your business flow needs it.

Note: do not send commands immediately and continuously right after `onServiceDiscovered()`. At that moment, the phone Bluetooth stack and the device-side GATT/notify state may not be fully ready, so commands sent too early can receive no response. Add a short delay or use a serial task queue before time sync, device-capability reads, and other initialization commands. If a command times out or receives no response, retry with a business-defined retry count and interval; only report disconnect or reconnect failure after retries are exhausted.

Special note: data request recommendation. After `onServiceDiscovered()` is reached, request `SetTimeReq` and `DeviceSupportReq.getReadInstance()` first. Wait until `SetTimeRsp` / `DeviceSupportFunctionRsp` is returned and support capabilities are cached before syncing or querying health data such as steps, sleep, heart rate, blood oxygen, HRV, pressure, and temperature. Do not start health-data requests before capability responses are available; otherwise the request may get no response, retry too often, or be incorrectly treated as unsupported while capability or GATT/notify state is still unknown.

Note: this is the dynamic registration point for SDK callback broadcasts. Do not rely only on a static `AndroidManifest.xml` receiver declaration. If `MyBluetoothReceiver` is not registered, common symptoms include missing connection-success events, no `onServiceDiscovered()` callback, commands being sent too early, or large-data sync initialization being skipped.

3.2 System Bluetooth State Receiver

`BluetoothReceiver` listens to system Bluetooth state, bond state, and ACL connection state. The Sample registers it dynamically after SDK initialization:

```
val deviceReceiver = BluetoothReceiver()
ContextCompat.registerReceiver(
    applicationContext,
    deviceReceiver,
    BleAction.getDeviceIntentFilter(),
    ContextCompat.RECEIVER_EXPORTED
)
```

`BleAction.getDeviceIntentFilter()` includes system Bluetooth and device-state actions such as:

- `BluetoothAdapter.ACTION_STATE_CHANGED`
- `BluetoothDevice.ACTION_BOND_STATE_CHANGED`
- `BluetoothDevice.ACTION_ACL_CONNECTED`
- `BluetoothDevice.ACTION_ACL_DISCONNECTED`

When Bluetooth is turned off, the Sample calls:

```
BleOperateManager.getInstance().setBluetoothTurnOff(false)
EventBus.getDefault().post(BluetoothEvent(false))
```

When Bluetooth is turned back on, it restores SDK Bluetooth state and attempts to reconnect with the cached device address:

```
BleOperateManager.getInstance().setBluetoothTurnOff(true)
BleOperateManager.getInstance().reConnectMac =
DeviceManager.getInstance().deviceAddress
BleOperateManager.getInstance().connectDirectly(DeviceManager.getInstance().deviceAddress)
```

You may keep boot/shutdown static receiver declarations in `AndroidManifest.xml`, but runtime Bluetooth state, bond-state, and ACL broadcasts should be dynamically registered in `Application` as shown in the Sample. This avoids missing broadcasts on different Android versions.

External apps should keep both receiver registrations and connection-state handling, then delay or queue time sync and device-capability reads after `MyBluetoothReceiver.onServiceDiscovered()`. If no response is received, retry instead of treating the first no-response result as unsupported.

The Sample `@BleIsConnected` pre-check now verifies that system Bluetooth is enabled before checking the SDK connection state. When Bluetooth is off, it shows "Please turn on Bluetooth first" and does not send the command. External apps should check both the system Bluetooth switch and the SDK connection state before executing any device command.

```
CommandHandle.getInstance()
    .executeReqCmd(SetTimeReq(1), ICommandResponse<SetTimeRsp> {
        // Cache SetTimeRsp capability fields.
    })

CommandHandle.getInstance()
    .executeReqCmd(
        DeviceSupportReq.getReadInstance(),
        ICommandResponse<DeviceSupportFunctionRsp> {
            // Cache DeviceSupportFunctionRsp capability fields.
        }
    )
```

Req/Rsp parameter reference

Type	Construction / field	Type / range	Description
<code>SetTimeReq</code>	<code>SetTimeReq(language)</code>	<code>Int</code>	Synchronizes phone time and language to the device. Use the language value required by the product protocol; the Sample uses 1.
<code>SetTimeRsp</code>	<code>mSupportTemperature</code> , <code>mSupportBloodOxygen</code> , <code>mSupportBloodPressure</code> , <code>mSupportPressure</code> , <code>mSupportHrv</code>	<code>Boolean</code>	Health capability flags. Prefer the matching <code>supports*()</code> helper when one is available.
<code>SetTimeRsp</code>	<code>mSupportPlate</code> , <code>mSupportCustomWallpaper</code> , <code>mMaxWatchFace</code>	<code>Boolean</code> , <code>Int</code>	Built-in/custom watch-face support and the maximum watch-face count.
<code>SetTimeRsp</code>	<code>mSupportMenstruation</code> , <code>mSupportFeature</code> , <code>mSupportOneKeyCheck</code> , <code>mSupportWeather</code> , <code>mNewSleepProtocol</code>	<code>Boolean</code>	Menstrual-cycle, fatigue, one-key check, weather, and new sleep protocol capabilities.
<code>SetTimeRsp</code>	<code>mSupportContact</code> , <code>mMaxContacts</code> , <code>mSupportManualHeart</code> , <code>mSupportManualBloodOxygen</code> , <code>mSupportAppMeasure</code>	<code>Boolean</code> , <code>Int</code>	Contact capacity and manual/App-started measurement capabilities. Check <code>mSupportAppMeasure</code> before using <code>RealTimeHeartRate</code> .
<code>SetTimeRsp</code>	<code>mSupportECard</code> , <code>mSupportLocation</code> , <code>mMusicSupport</code> , <code>mEbookSupport</code> , <code>mSupportWeChat</code> , <code>mSupportAvatar</code>	<code>Boolean</code>	E-card, location, music, ebook, WeChat, and avatar capabilities.
<code>SetTimeRsp</code>	<code>mSupportBloodSugar</code> , <code>bpSettingSupport</code> , <code>mYaweiSupport</code> , <code>mSupportRecord</code> , <code>rtkMcu</code>	<code>Boolean</code>	Blood-sugar, blood-pressure setting, diastolic/systolic calculation, sport-record, and RTK-MCU flags.

4. Device Connection and State

Sample files:

- `HomeActivity`
- `DeviceBindActivity`

4.1 Scan Devices

```
BleScannerHelper.getInstance()
    .scanDevice(context, null, scanCallback)

BleScannerHelper.getInstance().stopScan(context)
```

`DeviceBindActivity` filters by device name, sorts by RSSI, and connects directly after the user selects a device.

4.2 Connect, Disconnect, and Unbind

```
BleOperateManager.getInstance().connectDirectly(deviceAddress)
BleOperateManager.getInstance().connectWithScan(deviceAddress)
BleOperateManager.getInstance().disconnect()
BleOperateManager.getInstance().unBindDevice()
```

After connection succeeds, the Sample sends the bind-success command:

```
CommandHandle.getInstance()
    .executeReqCmd(SimpleKeyReq(Constants.CMD_BIND_SUCCESS), null)
```

After a successful connection, read and cache device capability, including `supportBlePair`. The Sample sends `DeviceSupportReq.getReadInstance()` after the connection event and uses `DeviceSupportFunctionRsp.supportBlePair` to decide whether the device requires or supports system Bluetooth pairing:

```
CommandHandle.getInstance()
    .executeReqCmd(
        DeviceSupportReq.getReadInstance(),
        ICommandResponse<DeviceSupportFunctionRsp> { rsp ->
            DeviceFunctionSupport.instance.updateFrom(rsp)
            if (rsp.supportBlePair) {
                BleOperateManager.getInstance().bleCreateBond()
            }
        }
    )
```

Handle disconnect and unbind according to `supportBlePair`:

- `supportBlePair = true`: the device uses the system Bluetooth pairing flow. When the user taps "disconnect", the Sample calls `unBindDevice()` to disconnect and remove the binding relationship, then refreshes the UI to the disconnected state.
- `supportBlePair = false`: when the user taps "disconnect", call `disconnect()` to close only the current BLE connection without clearing the binding relationship.
- When the user explicitly taps "unbind", call `unBindDevice()` regardless of `supportBlePair`, then refresh the UI to the disconnected state.

Req/Rsp parameter reference

Type	Construction / field	Type / range	Description
<code>SimpleKeyReq</code>	<code>SimpleKeyReq(Command)</code>	<code>Byte</code> ; <code>Constants.CMD_*</code>	Sends a command that has no additional payload, such as bind success or battery query. Use an SDK constant rather than a literal value.
<code>DeviceSupportReq</code>	<code>getReadInstance()</code>	No business parameter	Reads the extended device capability bitmap.
<code>DeviceSupportFunctionRsp</code>	<code>supportTouch</code> , <code>supportGesture</code> , <code>onlyTouch</code> , <code>tpsSleep</code>	<code>Boolean</code>	Touch, gesture, touch-only, and touch-panel sleep capabilities.
<code>DeviceSupportFunctionRsp</code>	<code>supportMoslin</code> , <code>supportworship</code> , <code>supportNewPraise</code> , <code>supportMoslinTouch</code>	<code>Boolean</code>	Muslim/prayer/praise capabilities and their touch-control support.
<code>DeviceSupportFunctionRsp</code>	<code>supportAPPRevision</code> , <code>supportBlePair</code> , <code>resumeServices</code>	<code>Boolean</code>	App-revision, system Bluetooth pairing, and service-resume capabilities.
<code>DeviceSupportFunctionRsp</code>	<code>supportRingMusic</code> , <code>supportRingVideo</code> , <code>supportRingEbook</code> , <code>supportRingCamera</code> , <code>supportRingPhoneCall</code> , <code>supportRingGame</code>	<code>Boolean</code>	Ring-side media, camera, call, and game features.
<code>DeviceSupportFunctionRsp</code>	<code>supportRingMusicTouch</code> , <code>supportRingVideoTouch</code> , <code>supportRingEbookTouch</code> , <code>supportRingCameraTouch</code> , <code>supportRingPhoneCallTouch</code> , <code>supportRingGameTouch</code>	<code>Boolean</code>	Touch-control support for the corresponding ring-side features.
<code>DeviceSupportFunctionRsp</code>	<code>supportHeart</code> , <code>supportHeartTouch</code> , <code>supportIntervalHeartRate</code> , <code>supportIntervalHeartRateRemind</code> , <code>supportIntervalBloodOxygen</code> , <code>supportIntervalTemperature</code>	<code>Boolean</code>	Heart-rate/touch and interval health-measurement capabilities.
<code>DeviceSupportFunctionRsp</code>	<code>supportLongSit</code> , <code>supportDrink</code> , <code>supportSkinTemperature</code> , <code>supportNoSingleTemperature</code> , <code>supportEcg</code> , <code>supportEmotion</code>	<code>Boolean</code>	Sedentary, drink reminder, skin temperature, single-temperature restriction, ECG, and emotion capabilities.
<code>DeviceSupportFunctionRsp</code>	<code>supportNotification</code> , <code>supportCallReminder</code> , <code>supportRingGestureDND</code> , <code>supportDoNotDisturb</code> , <code>supportAlarm</code>	<code>Boolean</code>	Notification, call reminder, gesture-DND, DND, and alarm capabilities.
<code>DeviceSupportFunctionRsp</code>	<code>supportRt11</code> , <code>deviceNoScreen</code> , <code>supportAiAnalyze</code> , <code>supportLoversSpace</code> , <code>supportFriends</code> , <code>supportLoverInteract</code> , <code>supportUltraviolet</code>	<code>Boolean</code>	Product/model and social/analysis/ultraviolet capabilities.

4.3 Bluetooth Pairing

The Sample already has an entry at `HomeActivity.fivBtPair`:

```
BleOperateManager.getInstance().bleCreateBond()
```

For devices with `supportBlePair = true`, the Sample automatically calls `bleCreateBond()` after connection succeeds and capability is read. The BT pairing entry on the home page can also trigger the same method manually. External apps should keep the system bond-state receiver so the UI and business flow can react to pairing success, cancellation, or failure.

4.4 Find Device, Battery, and Firmware Version

```
CommandHandle.getInstance().executeReqCmd(FindDeviceReq()) { }

CommandHandle.getInstance()
    .executeReqCmd(
        SimpleKeyReq(Constants.CMD_GET_DEVICE_ELECTRICITY_VALUE),
        ICommandResponse<BatteryRsp> { rsp ->
            val battery = rsp.batteryValue
            val charging = rsp.isCharging
        }
    )
```

Firmware and hardware versions are read through

`QCBluetoothCallbackCloneReceiver.onCharacteristicRead`. The Sample caches them in `MyBluetoothReceiver` and displays firmware version on the home and OTA pages.

Req/Rsp parameter reference

Type	Construction / field	Type / range	Description
<code>FindDeviceReq</code>	<code>FindDeviceReq()</code>	No business parameter	Makes the connected device identify itself.
<code>BatteryRsp</code>	<code>batteryValue</code> / <code>getBatteryValue()</code>	<code>Int</code> , 0..100	Remaining battery percentage.
<code>BatteryRsp</code>	<code>charging</code> / <code>isCharging()</code>	<code>Boolean</code>	Whether the device is currently charging.

5. Sample Feature Map

Sample Category	Activity	Main Capabilities
Device connection	<code>HomeActivity</code> , <code>DeviceBindActivity</code>	Scan, connect, reconnect, disconnect, unbind, BT pairing, find device, battery, firmware version

Sample Category	Activity	Main Capabilities
Health settings	HealthSettingsActivity	Steps, sleep, sport records, heart rate, heart-rate PPG, RRI auto measurement and block sync, blood pressure, blood oxygen, blood-oxygen PPG, HRV, stress/pressure, pressure interval setting, temperature, sugar/lipids, drink reminder, sedentary reminder, batch sync
Target and profile	TargetAndProfileActivity	Goals, time/unit format, user profile, Phoneld
Display	DisplayActivity	Brightness, display time, orientation, style, dial index, palm screen
Touch/Gesture	TouchGestureActivity	Parameterized touch, gesture, and RT11 touch-control read/write demo
Device	DeviceActivity	Temperature unit, battery saving, camera control, wear calibration, shutdown, reboot, restore factory settings
Notification	NotificationSettingActivity	ANCS switch, ANCS binding, DND, music switch, notification forwarding, notification event notify, custom light notification, custom vibration notification
Advanced	AdvancedSettingActivity	AGPS, Intell, Muslim/praise, device support, personalization setting, close device Bluetooth, shutdown
OTA	OtaActivity	Select firmware, BLE update, force update, progress and result

6. Health Settings

Sample files: `HealthSettingsActivity` and its sub-pages.

6.1 Steps

```

CommandHandle.getInstance()
    .executeReqCmd(
        SimpleKeyReq(Constants.CMD_GET_STEP_TODAY),
        ICommandResponse<TodaySportDataRsp> { rsp ->
            // rsp.totalSteps, rsp.calorie, rsp.walkDistance
        }
    )

```

```

CommandHandle.getInstance()
    .executeReqCmd(
        ReadDetailSportDataReq(dayOffset, 0, 95),
        ICommandResponse<ReadDetailSportDataRsp> { rsp ->
            // rsp.bleStepDetailses
        }
    )

// Activity score: get today's steps first, then calculate by age.
CommandHandle.getInstance()
    .executeReqCmd(
        SimpleKeyReq(Constants.CMD_GET_STEP_TODAY),
        ICommandResponse<TodaySportDataRsp> { rsp ->
            if (rsp.status == BaseRspCmd.RESULT_OK) {
                val score =
                    ActivityScoreUtils.calculateActivityScore(rsp.sportTotal.totalSteps, age)
            }
        }
    )

```

Req/Rsp parameter reference

Type	Construction / field	Type / range	Description
TodaySportDataRsp	sportTotal.daysAgo, year, month, day	Int	Day offset (0 today) and calendar date of the summary.
TodaySportDataRsp	totalSteps, runningSteps, calorie, walkDistance	Int; steps, cal, m	Daily steps, running steps, energy (small calories), and walking distance. Convert calories according to the product display convention.
TodaySportDataRsp	sportDuration, sleepDuration	Int; min	Daily activity and sleep duration.
ReadDetailSportDataReq	dayOffset	Int, 0..6	0 is today, 1 yesterday; the device retains up to seven days.
ReadDetailSportDataReq	startIndex, endIndex	Int, 0..95	Inclusive 15-minute slot range; a full day is 0, 95.
ReadDetailSportDataRsp	bleStepDetailses	List<BleStepDetails>	Returned detail points.
BleStepDetails	year, month, day, timeIndex	Int; timeIndex 0..95	Date and 15-minute slot; minute-of-day is timeIndex * 15.
BleStepDetails	walkSteps, runSteps, distance, calorie	Int; steps, m, cal	Walking/running steps, distance, and energy for the slot.
ActivityScoreUtils.calculateActivityScore(steps, age)	steps	Int, >= 0	Steps used for scoring, usually TodaySportDataRsp.sportTotal.totalSteps.
ActivityScoreUtils.calculateActivityScore(steps, age)	age	Int, >= 0	User age. The SDK converts steps to an activity score by age band, capped at 100.
ActivityScoreUtils.calculateActivityScore(steps, age)	return value	Int, 0..100	Activity score. Returns 0 when steps are <= 0 or age is <= 0.

6.2 Sleep

```

LargeDataHandler.getInstance().syncSleepList(
    0,
    ILargeDataSleepResponse { display ->
        // New sleep display data.
    },
    ILargeDataLaunchSleepResponse { launch ->
        // New-protocol sleep data.
    }
)

```

Req/Rsp parameter reference

Type	Field	Type / range	Description
SleepNewProtoResp	dayIndex	Int, 0..6	0 is today and 1 is yesterday; use only the device retention window.
SleepNewProtoResp	st, et	Int, Unix seconds	Main sleep start and end time. Convert to local time only for display.
SleepNewProtoResp	list	List<DetailBean>	Ordered main-sleep segments.
SleepNewProtoResp	lunchBreak, lunchSt, lunchEt	Boolean, Unix seconds	Whether nap data is present and its start/end time.
SleepNewProtoResp.DetailBean	d	Int, min	Duration of this sleep segment.
SleepNewProtoResp.DetailBean	t	Int, 1..5	1 deep, 2 light, 3 awake, 4 REM, 5 no sleep/not worn.
SleepDisplay	sleepEfficiency / getSleepEfficiency()	Int, 0..100	Sleep efficiency, displayed as a percentage. Returns 0 when there is no valid sleep.
SleepDisplay	sleepScore / getSleepScore()	Int, 0..100	Sleep score calculated by the SDK from display data such as total sleep, deep sleep, and light sleep.
LongSitResp	index	Int	Day offset: 0 today, 1 yesterday, and so on.
LongSitResp	list	List<DetailBean>	Sedentary-state segments for the selected day.
LongSitResp.DetailBean	d, t	Int; t=0..2	Duration plus state: 0 static (fewer than 30 steps/min), 1 sedentary triggered, 2 moving.

6.3 Heart Rate, Blood Pressure, Blood Oxygen, HRV, Pressure, and Temperature

The current Sample health sub-pages mainly provide these entries:

- Setting read, write, explicit open, and explicit close.
- Today, specific-day, and day-range health-data queries.
- Manual measurement start/stop.
- Pages that support PPG also provide raw PPG start/stop entries.

Heart rate, blood pressure, blood oxygen, HRV, pressure, and temperature automatic-measurement settings can all be enabled or disabled with the `enable` parameter. In the Sample, "Settings Write" and "Open Setting" write `enable = true`, while "Close Setting" writes `enable = false`; keep the other protocol-required fields, such as intervals, time windows, or reminder thresholds, even when disabling a setting.

Heart-rate setting example:

```
CommandHandle.getInstance()
    .executeReqCmd(
        HeartRateSettingReq.getReadInstance(),
        ICommandResponse<HeartRateSettingRsp> { rsp ->
            // rsp.isEnabled, rsp.startInterval, rsp.tooLowReminder,
            rsp.tooHighReminder
        }
    )

CommandHandle.getInstance()
    .executeReqCmd(
        HeartRateSettingReq.getWriteInstance(enable, 10, 10, 40, 110),
        ICommandResponse<HeartRateSettingRsp> { }
    )
```

Other health setting switch examples:

```
BpSettingReq.getWriteInstance(enable, StartEndTimeEntity(9, 0, 18, 0), 60)
BloodOxygenSettingReq.getWriteInstance(enable, 2.toByte())
HrvSettingReq.getWriteInstance(enable)
PressureSettingReq.getWriteInstance(enable)
SugarLipidsSettingReq.getWriteTemperatureInstance(enable, 2)
```

Manual measurement example:

```
BleOperateManager.getInstance().manualModeHeart({ rsp ->
    // rsp.value
}, false)

BleOperateManager.getInstance().manualModeSpO2({ rsp ->
    // rsp.value
}, false)
```

Heart-rate raw PPG example:

```

BleOperateManager.getInstance().manualModeHeartRateRawData(
    ICommandResponse<StopHeartRateRsp> { rsp ->
        // rsp.ppgCount, rsp.greenLightPpg, rsp.heartRate, rsp.rri
    },
    25,
    false
)

BleOperateManager.getInstance().manualModeHeartRateRawData(
    null,
    25,
    true
)

```

Blood-oxygen raw PPG example:

```

BleOperateManager.getInstance().manualModeBloodOxygenRawData(
    ICommandResponse<StopHeartRateRsp> { rsp ->
        // rsp.ppgCount, rsp.redLightPpg, rsp.infraredPpg, rsp.bloodOxygen
    },
    25,
    false
)

BleOperateManager.getInstance().manualModeBloodOxygenRawData(
    null,
    25,
    true
)

```

If you need PPG-related fields together with a manual measurement result, use `manualModeHeartPPG(...)` and `manualModeBloodOxygenPPG(...)`; pass `true` as the third argument to stop measurement.

Real-time heart-rate measurement example:

Reference usage:

D:\Project\Android\QRing_004\app\src\main\java\com\qcwireless\qcwatch\ui\home\heart\HeartActivity.kt sends `RealTimeHeartRate(1)` from the start button to start real-time heart rate. Note: the first callback after start may return `heart=0` or no valid heart-rate value, so do not request only once and do not wait too long before continuing. However, only start scheduled `RealTimeHeartRate(3)` requests after `RealTimeHeartRate(1)` succeeds. While the page remains in measuring state, use a `Handler` to quickly keep sending `RealTimeHeartRate(3)`, for example poll every 2 seconds. When the callback returns `RealTimeHeartRateRsp` and `heart > 0`, `heart` is the current real-time heart-rate value and can be displayed, for example in `tvCurrHeart`. When the page pauses, finishes, the user taps stop, or a request times out, send `RealTimeHeartRate(2)` to stop real-time heart rate and remove the scheduled task so the page does not keep requesting in the background.

```

val handler = Handler(Looper.getMainLooper())
var realtimeHeartRateRunning = false

val continueRealtimeHeartRateTask = object : Runnable {

```

```

        override fun run() {
            if (!realtimeHeartRateRunning) return
            CommandHandle.getInstance()
                .executeReqCmd(
                    RealTimeHeartRate(0x03), // continue/refresh real-time heart rate
                    ICommandResponse<RealTimeHeartRateRsp> { rsp ->
                        val continueSuccess = rsp != null && rsp.status ==
BaseRspCmd.RESULT_OK
                        if (continueSuccess && rsp.heart > 0) {
                            runOnUiThread {
                                tvCurrHeart.text = rsp.heart.toString()
                            }
                        }
                        if (continueSuccess) {
                            handler.postDelayed(this, 2_000L)
                        } else {
                            stopRealtimeHeartRate()
                        }
                    }
                )
        }
    }

    fun startRealtimeHeartRate() {
        if (!setTimeRsp.mSupportAppMeasure) return
        realtimeHeartRateRunning = true
        CommandHandle.getInstance()
            .executeReqCmd(
                RealTimeHeartRate(0x01), // start real-time heart rate
                ICommandResponse<RealTimeHeartRateRsp> { rsp ->
                    val startSuccess = rsp != null && rsp.status ==
BaseRspCmd.RESULT_OK
                    if (startSuccess && rsp.heart > 0) {
                        runOnUiThread {
                            tvCurrHeart.text = rsp.heart.toString()
                        }
                    }
                    if (startSuccess) {
                        handler.postDelayed(continueRealtimeHeartRateTask, 2_000L)
                    } else {
                        realtimeHeartRateRunning = false
                        handler.removeCallbacks(continueRealtimeHeartRateTask)
                    }
                }
            )
    }

    fun stopRealtimeHeartRate() {
        realtimeHeartRateRunning = false
        handler.removeCallbacks(continueRealtimeHeartRateTask)
        CommandHandle.getInstance()
            .executeReqCmdNoCallback(RealTimeHeartRate(0x02)) // stop real-time heart
rate
    }

```

```
// Call stopRealtimeHeartRate() from onPause/onDestroy/the stop button/timeout handling.
```

Req/Rsp parameter reference

Request type	Construction / parameter	Type / range	Description
HeartRateSettingReq	getReadInstance()	No business parameter	Reads automatic heart-rate settings.
HeartRateSettingReq	getWriteInstance(enable, interval[, startInterval, low, high[, mainSwitch]])	Boolean, Int	Writes the switch, measurement interval, optional start interval, low/high reminder thresholds, and main switch. Preserve device-supported values when disabling.
BloodOxygenSettingReq	getReadInstance() / getWriteInstance(enable[, interval])	Boolean, Byte, min	Reads or writes the blood-oxygen automatic measurement switch and optional interval.
BpSettingReq	getReadInstance() / getWriteInstance(enable, timeRange, multiple)	Boolean, StartEndTimeEntity, Int	Blood-pressure switch, active start/end time, and measurement period; the legacy default period is 60 minutes.
HrvSettingReq	getReadInstance() / constructor or getWriteInstance(enable)	Boolean, optional Int interval	Reads or writes HRV automatic measurement.
SugarLipidsSettingReq	getReadTemperatureInstance() / getWriteTemperatureInstance(enable[, interval])	Boolean, Int, min	Reads or writes the current temperature automatic-measurement switch and optional interval.
SugarLipidsSettingReq	getReadRriInstance() / getWriteRriInstance(enable)	Boolean	Reads or writes RRI automatic measurement.
SugarLipidsSettingReq	getReadEmotionInstance() / getWriteEmotionInstance(enable)	Boolean	Reads or writes emotion automatic measurement.
SugarLipidsSettingReq	generic getReadInstance(type) / getWriteInstance(type, enable, value[, interval])	Byte, Boolean, Int	Low-level form. Prefer TYPE_TEMPERATURE, TYPE_RRI, TYPE_EMOTION and the typed factories above.
ReadHeartRateReq	ReadHeartRateReq(utcTime)	Long, Unix seconds	Starting timestamp for legacy heart-rate history synchronization.
BpReadConformReq	BpReadConformReq(confirm)	Boolean	Confirms successful timed blood-pressure synchronization; confirmed records may be deleted by the device.
ReadPressureReq	ReadPressureReq(utcTime)	Long, Unix seconds	Starting timestamp for manual blood-pressure record synchronization.
HRVReq	HRVReq(dayOffset)	Byte, 0..6	Reads HRV data for the selected retained day.
StartHeartRateReq	getSimpleReq(type[, sampleRate])	Byte, optional Int	Starts the selected manual measurement; use TYPE_* constants. sampleRate is used by raw-data modes.
StartHeartRateReq	getRealtimeHeartRate(action)	Byte, ACTION_*	Starts, pauses, continues, or stops real-time heart rate.
RealTimeHeartRate	RealTimeHeartRate(type)	Int, 0x01 start, 0x02 stop, 0x03 continue	Controls real-time heart-rate measurement. Use only when SetTimeRsp.mSupportAppMeasure == true.

Response type	Field	Type / range	Description
HeartRateSettingRsp	enable, heartInterval, startInterval, tooLowReminder, tooHighReminder, mainSwitch	Boolean, Int	Current automatic heart-rate configuration and reminder thresholds.
BloodOxygenSettingRsp	enable, interval	Boolean, Byte, min	Automatic blood-oxygen switch and interval.
BpSettingRsp	enable, startEndTimeEntity, multiple	Boolean, time range, Int min	Automatic blood-pressure switch, active time window, and period.

Response type	Field	Type / range	Description
HRVSettingRsp	enable	Boolean	HRV automatic measurement switch.
BloodSugarLipidsSettingRsp	type, enable, value, startInterval, interval, read	Byte, Boolean, Int	Shared temperature/RRI/emotion setting result; interpret value fields according to type. read identifies a read response.
BloodSugarLipidsSettingRsp	rriAutoMeasureEnable	Boolean	RRI automatic measurement state.
ReadHeartRateRsp	dayIndex, mUtcTime, interval, range, endFlag	Int, Unix seconds, min, Boolean	Day/timestamp, sampling interval/range, and transfer completion flag.
ReadHeartRateRsp	mHeartRateArray, array	ByteArray / List<Int>	Legacy compact and minute-indexed heart-rate samples; use the returned interval when mapping indexes to time.
BpDataRsp	bpDataEntity.year, month, day, timeDelay, bpValues	Int, list	Timed blood-pressure date, delay/period, and values. Each value supplies minute-of-day and measured value.
ReadBlePressureRsp	valueList[].time, dbp, sbp	Unix seconds, mmHg	Manual blood-pressure record timestamp, diastolic pressure, and systolic pressure.
HRVRsp	offset, endFlag, hrvArray, range, today	Int, Boolean, ByteArray, date	Day offset, completion flag, HRV samples, sample range, and date. Legacy display values are raw value divided by 10.
StartHeartRateRsp	type, errCode, value, sbp, dbp	Byte / Int	Measurement type, result code (0 normal), generic value, and blood-pressure values.
StartHeartRateRsp	ppgCount, greenLightPpgL/H, redLightPpgL/H, infraredPpgL/H	Int	Raw PPG sequence and low/high words for green, red, and infrared channels.
StartHeartRateRsp	xL/xH, yL/yH, zL/zH	Int	Low/high words of accelerometer axes accompanying raw PPG data.
StartHeartRateRsp	heartRate, bloodOxygen, rri, heart, hrv, stress, temperature, intervalTemperature	Int / temperature object	Mode-specific calculated health values. Read only fields applicable to type.
RealTimeHeartRateRsp	heart	Int	Current real-time heart-rate value. For RealTimeHeartRate(0x01), display it when heart > 0.
StopHeartRateRsp	all StartHeartRateRsp result fields	Same as above	Final/manual/raw measurement result. It additionally exposes combined greenLightPpg, redLightPpg, and infraredPpg as Long.

6.4 RRI

RRI supports auto-measurement switch control, block-list query, and selected block sync.

```
CommandHandle.getInstance()
    .executeReqCmd(
        SugarLipidsSettingReq.getWriteRriInstance(true),
        ICommandResponse<BloodSugarLipidsSettingRsp> { rsp ->
            // rsp.isRriAutoMeasureEnable()
        }
    )

LargeDataHandler.getInstance().syncRriBlockList(
    DeviceManager.getInstance().deviceAddress,
    object : LargeDataHandler.IRriBlockListCallback {
        override fun onRriBlockList(blockList:
List<LargeDataHandler.RriBlockDescription>?) {
            // block.blockId, block.startMinuteSec, block.endMinuteSec
        }
    }
)

LargeDataHandler.getInstance().syncRriBlock(
    DeviceManager.getInstance().deviceAddress,
    blockId,
    object : LargeDataHandler.IRriBlockSyncCallback {
        override fun onRriBlockProgress(blockId: Int, current: Int, total: Int) {
        }

        override fun onRriBlockData(
            blockId: Int,
            rawBytes: ByteArray?,
            minuteData: List<LargeDataHandler.RriMinuteData>?
        ) {
            // minute.minuteStartSec, minute.rri
        }

        override fun onError(code: Int, message: String?) { }
    }
)
```

Parameter ranges and constraints:

Parameter / Field	Range / Source	Notes
<code>enable</code>	<code>true</code> / <code>false</code>	<code>true</code> enables RRI auto measurement; <code>false</code> disables it.
<code>deviceAddress</code>	Current connected device MAC	Use the SDK cached address, for example <code>DeviceManager.getInstance().deviceAddress</code> .
<code>blockId</code>	Returned by <code>syncRriBlockList</code>	Do not hard-code or infer IDs; only request IDs returned by the device.
<code>RRI_BLOCK_COUNT</code>	40	SDK constant for the maximum retained block count.
<code>RRI_BLOCK_SIZE</code>	4096 bytes	SDK constant for one RRI block.

Parameter / Field	Range / Source	Notes
<code>RRI_FRAGMENT_REQUEST_LEN</code>	1000 bytes	SDK constant for fragment requests.
<code>startMinuteSec</code> , <code>endMinuteSec</code> , <code>minuteStartSec</code>	Unix timestamp in seconds	Convert to local time only for display.
<code>rri</code>	List of RRI interval values	Values are returned by firmware per minute.

6.5 Pressure

Pressure supports setting read/write, interval writes on interval-capable devices, today/single-day/range health queries, and manual measurement start/stop. The current Sample has removed the standalone sync button; the page uses the shared health-query entries to call

`BleOperateManager.getInstance().getPressure(0, ...)` and related health-data query paths.

```

CommandHandle.getInstance()
    .executeReqCmd(
        PressureSettingReq.getReadInstance(),
        ICommandResponse<PressureSettingRsp> { rsp ->
            // rsp.isEnabled(), rsp.isIntervalSettingSupported(),
            rsp.getMinInterval(), rsp.getInterval()
        }
    )

// write the pressure automatic-measurement switch only.
CommandHandle.getInstance()
    .executeReqCmd(PressureSettingReq.getWriteInstance(enable)) { }

// On interval-capable devices, the sample caches interval fields before building
the write request.
PressureSettingReq.cacheIntervalSettings(minInterval, interval)
val writeReq = PressureSettingReq.getWriteInstance(enable).also {
    PressureSettingReq.clearCachedIntervalSettings()
}
CommandHandle.getInstance().executeReqCmd(writeReq) { }

BleOperateManager.getInstance().getTodayPressure(
    BleOperateManager.HealthDataCallback { rsp ->
        // rsp.getPressureArray(), rsp.getOffset(), rsp.isEndFlag(),
        rsp.getRange(), rsp.getToday()
    }
)

BleOperateManager.getInstance().getPressure(
    0,
    BleOperateManager.HealthDataCallback { rsp ->
        // dayIndex = 0 means today.
    }
)

BleOperateManager.getInstance().getPressures(
    0,
    7,

```

```

        BleOperateManager.HealthDataCallback { days ->
            // days: List<DayIndexedData<PressureRsp>>
        }
    )

    BleOperateManager.getInstance().manualModePressure(
        ICommandResponse<StartHeartRateRsp> { rsp ->
            // rsp.getValue()
        },
        false
    )

    BleOperateManager.getInstance().manualModePressure(
        ICommandResponse<StartHeartRateRsp> { rsp ->
            // rsp.status
        },
        true
    )

```

Parameter ranges and constraints:

Parameter / Field	Range / Source	Notes
<code>enable</code>	<code>true</code> / <code>false</code>	Pressure automatic measurement switch.
<code>supportsInterval</code>	Returned by <code>PressureSettingRsp.isIntervalSettingSupported()</code>	Show interval selection only when this is <code>true</code> ; otherwise write the switch only.
<code>minInterval</code>	Returned by <code>PressureSettingRsp.getMinInterval()</code>	Use the value returned by the device when interval setting is supported.
<code>interval</code>	<code>interval >= minInterval</code>	Sample candidates are 5/10/15/20/30/60 min, filtered by <code>minInterval</code> ; if all are smaller, it falls back to <code>minInterval</code> .
<code>PressureSettingReq.cacheIntervalSettings(minInterval, interval)</code>	Call before building an interval write request	Call <code>clearCachedIntervalSettings()</code> after <code>getWriteInstance(enable)</code> is built so later switch-only writes do not reuse a stale interval.
<code>dayIndex</code>	0 today, 1 yesterday, and so on	Used by <code>getPressure(dayIndex, callback)</code> .
<code>startDayIndex, count</code>	Sample example: 0, 7	Used by <code>getPressures(startDayIndex, count, callback)</code> ; keep it within the device-side retention window.
PressureRsp data	<code>getPressureArray()</code> / <code>getOffset()</code> / <code>isEndFlag()</code> / <code>getRange()</code> / <code>getToday()</code>	Returned by pressure health-data queries.
<code>isStop</code>	<code>false</code> starts, <code>true</code> stops	Used by <code>manualModePressure(callback, isStop)</code> .
Historical pressure window	7 days by default	See the device-side health data retention table below.

Req/Rsp parameter reference

Type	Construction / field	Type / range	Description
<code>PressureSettingReq</code>	<code>getReadInstance()</code>	No business parameter	Reads pressure automatic measurement settings.
<code>PressureSettingReq</code>	<code>getWriteInstance(enable[, interval])</code>	<code>Boolean</code> , <code>Int</code> , min	Writes the switch and optional interval. <code>cacheIntervalSettings(minInterval, interval)</code> is a compatibility path; clear it after building the request.
<code>PressureSettingRsp</code>	<code>enable</code> , <code>minInterval</code> , <code>interval</code> , <code>intervalSettingSupported</code>	<code>Boolean</code> , <code>Int</code> , min	Current switch, device minimum interval, selected interval, and whether interval writes are supported.

Type	Construction / field	Type / range	Description
PressureReq	PressureReq(dayOffset)	Byte, 0..6	Legacy pressure query for a retained day.
PressureRsp	offset, endFlag, pressureArray, range, today	Int, Boolean, ByteArray, date	Day offset, completion flag, pressure samples, sampling range, and date. Legacy display values are raw value divided by 10.

6.6 Emotion

Emotion supports switch read/write through `SugarLipidsSettingReq` and health-data queries for today, a specific day, or a day range. The Sample page uses the shared health query UI: today maps to `dayIndex = 0`, a specific day uses the selected `dayIndex`, and a range uses `startDayIndex` plus `count`.

```

CommandHandle.getInstance()
    .executeReqCmd(
        sugarLipidsSettingReq.getReadEmotionInstance(),
        ICommandResponse<BloodSugarLipidsSettingRsp> { rsp ->
            // rsp.isEnabled(), rsp.getType(), rsp.getInterval()
        }
    )

CommandHandle.getInstance()
    .executeReqCmd(
        sugarLipidsSettingReq.getWriteEmotionInstance(true),
        ICommandResponse<BloodSugarLipidsSettingRsp> { }
    )

BleOperateManager.getInstance().getEmotion(
    0,
    BleOperateManager.HealthDataCallback { rsp ->
        // dayIndex = 0 means today.
        // rsp.status is the emotion status.
    }
)

BleOperateManager.getInstance().getEmotions(
    0,
    7,
    BleOperateManager.HealthDataCallback { days ->
        // days: List<DayIndexedData<EmotionRsp>>
    }
)

```

Emotion status values:

status	Meaning
0	null
1	Excited
2	Happy

status	Meaning
3	Pleased
4	Calm
5	Nervous
6	Confused
7	Challenged
8	Uncomfortable
9	Sad
10	Anxious

Parameter ranges and constraints:

Parameter / Field	Range / Source	Notes
enable	true / false	Emotion automatic measurement switch.
SugarLipidsSettingReq.TYPE_EMOTION	SDK-defined emotion setting type	Prefer <code>getReadEmotionInstance()</code> and <code>getWriteEmotionInstance(enable)</code> instead of passing the type manually.
dayIndex	0 today, 1 yesterday, and so on	Used by <code>getEmotion(dayIndex, callback)</code> .
startDayIndex, count	Sample example: 0, 7	Used by <code>getEmotions(startDayIndex, count, callback)</code> ; keep it within the device-side retention window.
status	0..10	Emotion state returned by emotion health-data queries.

EmotionRsp response fields:

Field	Type / range	Description
offset, endFlag, range, today	Int, Boolean, Int, date	Day offset, transfer completion flag, sampling range, and date.
emotionArray	ByteArray	Compact emotion samples returned by the protocol.
valueList[].value, valueList[].status	Int; status 0..10	Sample value/time position and decoded emotion status.

6.7 Batch Sync and Task Chain

`HealthSettingsActivity.startDebugSyncAllChain()` demonstrates a task chain with:

- Command tasks: steps, heart rate, HRV, pressure.
- File tasks: temperature file data.
- Large-data tasks: blood oxygen, sleep, sport records.

The task chain also demonstrates three health-data query granularities:

```
// Today
getTodayHeartRate(callback)
getTodayBloodOxygen(callback)
getTodaySleep(callback)

// Specific dayIndex, where 0 means today and 1 means yesterday
getHeartRate(dayIndex, callback)
getHrv(dayIndex, callback)
getPressure(dayIndex, callback)
getTemperature(dayIndex, callback)
getBloodOxygen(dayIndex, callback)
getSleep(dayIndex, callback)
getSport(dayIndex, callback)

// Range query. The sample uses 0..6, for 7 days total.
getHeartRates(0, 7, callback)
getHrvs(0, 7, callback)
getPressures(0, 7, callback)
getTemperatures(0, 7, callback)
getBloodOxygens(0, 7, callback)
getSleeps(0, 7, callback)
getSports(0, 7, callback)
```

`getSport(...)` / `getSports(...)` returns sport records as `SportPlusEntity`:

Field	Type / unit	Description
<code>mSportType</code>	<code>Int</code>	Sport type index.
<code>mStartTime</code>	<code>Int</code> , Unix seconds	Sport start time.
<code>mDuration</code>	<code>Int</code> , seconds	Sport duration.
<code>mDistance</code>	<code>Int</code> , meters	Sport distance.
<code>mCalories</code>	<code>Float</code> , small calories	Calories.
<code>mSpeedAvg</code>	<code>Int</code> , cm/s	Average speed.
<code>mSpeedMax</code>	<code>Int</code> , cm/s	Maximum speed.
<code>mRateAvg</code>	<code>Int</code> , beats/minute	Average heart rate.
<code>mRateMin</code>	<code>Int</code> , beats/minute	Minimum heart rate.

Field	Type / unit	Description
<code>mRateMax</code>	<code>Int</code> , beats/minute	Maximum heart rate.
<code>mElevation</code>	<code>Int</code> , cm	Average altitude.
<code>mUphill</code>	<code>Int</code> , cm	Cumulative climb.
<code>mDownhill</code>	<code>Int</code> , cm	Cumulative descent.
<code>mStepRate</code>	<code>Int</code> , steps/minute	Average cadence.
<code>mSportCount</code>	<code>Int</code> , times	Number of sport sessions.
<code>steps</code>	<code>Int</code> , steps	Sport steps.
<code>mLocations</code>	<code>List<SportLocation></code>	Sport location/track point list.
<code>SportLocation.mRateReal</code>	<code>Int</code> , beats/minute	Real-time heart rate for the track point.

When `largeData(...)`, `file(...)`, `measurement(...)`, `task(...)`, or `dfu(...)` is used inside a chain, the callback must still call `task.success(...)` or `task.fail(...)` to finish the current step.

6.8 Device-Side Health Data Retention

When implementing historical data sync, catch-up reads, or task-chain retries, use the table below as the default query-window reference. The actual synced result still depends on device firmware, device capability, and the API response.

Health Data Type	Device-Side Fetch Range	Notes
Activity steps	7 days	
Sleep	7 days	
Sport records	Latest 7 records	Retained by record count
Heart rate	7 days	
One-minute heart rate	3 days	
Pressure	7 days	
Blood oxygen	3 days	
One-minute blood oxygen	7 days	
HRV	7 days	
Blood pressure	7 days	Reports only once
Temperature	7 days	

Health Data Type	Device-Side Fetch Range	Notes
One-minute temperature	7 days	
Manual body temperature	Stored in the app	Not fetched from a device-side historical window
Three-value temperature	7 days	
Blood glucose	7 days	

Sync device-side data periodically according to your product needs. For data outside the device retention window, rely on values already persisted by the app or backend service.

7. Target and Profile

Sample file: `TargetAndProfileActivity`.

7.1 Goals

```

CommandHandle.getInstance().executeReqCmd(
    TargetSettingReq.getWriteInstance(
        8000, 500, 3000, 60, 600
    ),
    null
)

CommandHandle.getInstance()
    .executeReqCmd(TargetSettingReq.getReadInstance()) { rsp ->
        // Read target values.
    }

```

Parameters:

- `step`: step goal.
- `calorie`: calorie goal.
- `distance`: distance goal in meters.
- `sportMinute`: exercise-duration goal in minutes.
- `sleepMinute`: sleep-duration goal in minutes.

Req/Rsp parameter reference

Type	Construction / parameter	Type / unit	Description
<code>TargetSettingReq</code>	<code>getReadInstance()</code>	No business parameter	Reads current goals.

Type	Construction / parameter	Type / unit	Description
TargetSettingReq	<code>getWriteInstance(step, calorie, distance[, sportMinute, sleepMinute])</code>	Int ; steps, cal, m, min	Writes step, energy, distance, exercise-duration, and sleep-duration goals. The legacy protocol sends displayed kcal multiplied by 1000.

7.2 Time Format, Unit, and User Profile

```

CommandHandle.getInstance().executeReqCmd(
    TimeFormatReq.getWriteInstance(
        true, 0, 0, 30, 170, 65, 115, 75, 160
    )
) { }

CommandHandle.getInstance()
    .executeReqCmd(TimeFormatReq.getReadInstance()) { rsp ->
        // Read time format, unit, and user profile.
    }

```

Req/Rsp parameter reference

Type	Construction / field	Type / range	Description
TimeFormatReq	<code>getReadInstance()</code>	No business parameter	Reads time/unit/profile settings.
TimeFormatReq	<code>getWriteInstance(is24Hour, ...)</code>	Boolean plus profile values	Writes 12/24-hour mode and the selected overload's unit/profile fields. Use the overload matching the firmware and preserve values not edited by the UI.
TimeFormatReq	common profile values	sex, age, height, weight, stride and related Int fields	Values are product-profile data; validate them against the app's profile limits before sending.

Type	Construction / field	Type / range	Description
TimeFormatRsp	is24, metric	Boolean	true means 24-hour time / metric units.

7.3 PhoneId

```

CommandHandle.getInstance()
    .executeReqCmd(PhoneIdReq.getWriteInstance("123456789123")) { }

CommandHandle.getInstance()
    .executeReqCmd(PhoneIdReq.getReadInstance()) { rsp ->
        val userId = rsp.getUserId()
    }

```

Req/Rsp parameter reference

Type	Construction / field	Type / range	Description
PhoneIdReq	getReadInstance()	No business parameter	Reads the bound user identifier.
PhoneIdReq	getWriteInstance(userId)	String; Sample uses 12 digits	Writes the product user identifier; use a stable value for the same account.
PhoneIdRsp	userId / getUserId()	String	Identifier currently stored by the device.

8. Display and Device

Sample files:

- Display: DisplayActivity
- Device: DeviceActivity

8.1 Display Settings

The Sample covers:

- BrightnessSettingsReq
- DisplayClockReq
- DisplayOrientationReq
- DisplayStyleReq
- DisplayTimeReq
- DialIndexReq
- PalmScreenReq

- `TouchControlReq`

Palm screen example:

```
CommandHandle.getInstance()
    .executeReqCmd(PalmScreenReq.getWriteInstance(true, true), null)

CommandHandle.getInstance()
    .executeReqCmd(PalmScreenReq.getRingReadInstance()) { rsp ->
        // rsp.isEnabled, rsp.isLeft, rsp.screenLight, rsp.start, rsp.end
    }

CommandHandle.getInstance()
    .executeReqCmd(TouchControlReq.getWriteInstance(9, true, 5), null) // Open
touch

CommandHandle.getInstance()
    .executeReqCmd(TouchControlReq.getWriteInstance(0, true, 5), null) // Close
touch

CommandHandle.getInstance()
    .executeReqCmd(TouchControlReq.getWriteInstance(9, false, 5), null) // Open
gesture

CommandHandle.getInstance()
    .executeReqCmd(TouchControlReq.getWriteInstance(0, false, 5), null) // Close
gesture
```

For `TouchControlReq.getWriteInstance(type, isTouch, strength)`, `type = 9` opens the control and `type = 0` closes it. `isTouch = true` controls touch, while `isTouch = false` controls gesture. The Sample uses `strength = 5` by default and checks `supportTouch` or `supportGesture` before sending the command.

Req/Rsp parameter reference

Type	Construction / field	Type / range	Description
<code>BrightnessSettingsReq</code>	<code>getReadInstance()</code> / <code>getWriteInstance(brightness)</code>	<code>Int</code> , device-supported level	Reads or writes display brightness.
<code>DisplayClockReq</code>	<code>getReadInstance()</code> / <code>getWriteInstance(enable)</code>	<code>Boolean</code>	Reads or writes clock display.
<code>DisplayOrientationReq</code>	<code>getReadInstance()</code> / <code>getWriteInstance(firstFlag, secondFlag)</code>	<code>Boolean</code>	Reads or writes the two orientation/handedness flags defined by the firmware. Preserve the Sample mapping for the target product.
<code>DisplayStyleReq</code>	<code>getReadInstance()</code> / <code>getWriteInstance(style)</code>	<code>Int</code> , device-supported index	Reads or writes display style.
<code>DisplayTimeReq</code>	<code>getReadInstance()</code> / <code>getWriteInstance(startHour, startMinute, endHour, endMinute, duration)</code>	<code>Int</code> ; hour/minute	Reads or writes the display active window and screen-on duration.

Type	Construction / field	Type / range	Description
DisplayTimeReq	getWriteInstanceNew(..., enable) / getDeleteInstance()	values above plus Boolean	Writes the extended schedule or deletes it.
DialIndexReq	getReadInstance() / getWriteInstance(index)	Int, installed dial index	Reads or selects the current dial.
PalmScreenReq	getReadInstance() / getRingReadInstance()	No business parameter	Reads raise-to-wake settings; use the ring- specific factory for ring firmware.
PalmScreenReq	getWriteInstance(enable, left)	Boolean	Writes raise-to-wake and left/right-hand selection.
PalmScreenRsp	enable, left, needPalm, screenLight, maxLight, dnd, start, end	Boolean, Int	Raise-to-wake state/hand, palm requirement, brightness limits, DND state, and active window.
TouchControlReq	getReadInstance(isTouch) / getWriteInstance(type, isTouch, strength[, sleepTime])	Boolean, Int	Reads or writes normal touch and gesture settings. The normal-touch four- argument overload carries the sleep time.
TouchControlReq	getWriteTpSleepInstance(appType, state)	Int	Writes RT11 touch-panel application state; SDK 1.0.0.53 has no separate sleepTime parameter for this factory.
TouchControlRsp	isRead(), isTouch(), getAppType(), getStrength(), getSleepTime(), isTouchSleep()	Boolean, Int	Returns read/write mode, touch/gesture target, function type, strength, sleep time, and sleep state.

8.2 Touch/Gesture

Sample file: `TouchGestureActivity`. The Home page opens it from the Touch/Gesture entry, and the old fixed touch/gesture switches have been consolidated into this page.

This page documents SDK factory parameters instead of raw byte fields. It keeps all `appType` function options visible. Before sending a command, it only checks the selected top-level capability: touch checks `supportTouch`, gesture checks `supportGesture`, and RT11 checks `supportRt11` & `tpsSleep`. Unsupported commands are blocked with a Toast prompt.

Parameter	Page menu / SDK usage	Values	Description
type	Top-level function type	0 touch, 1 gesture, 2 RT11 touch	Selects which SDK factory to call and which capability flag to check.
isTouch	TouchControlReq.getReadInstance(isTouch) / getWriteInstance(appType, isTouch, ...)	true / false	true means touch, false means gesture; RT11 writes do not use this parameter.
appType	Touch/gesture function type	See table below	Target function to set, or close.
strength	Gesture strength	1-10	Gesture writes use TouchControlReq.getWriteInstance(appType, false, strength); normal touch writes also keep this parameter.

Parameter	Page menu / SDK usage	Values	Description
<code>sleepTime</code>	Touch auto-sleep time	1-5 minutes	Normal touch writes pass this through the fourth argument of <code>TouchControlReq.getWriteInstance(appType, true, strength, sleepTime)</code> .
<code>state</code>	RT11 touch state	0 awake, 1 sleeping	RT11 writes use <code>TouchControlReq.getWriteTpSleepInstance(appType, state)</code> . SDK 1.0.0.53 does not expose a separate RT11 <code>sleepTime</code> parameter.

<code>appType</code>	Function
0	Close
1	HID music control
2	HID short-video control
3	Muslim/praise counter
4	HID e-book control
5	Camera control
6	Phone-call control
7	Game control
8	Manual heart-rate measurement
9	Touch event report
10	Lover interaction report

```
// Read touch
CommandHandle.getInstance()
    .executeReqCmd(TouchControlReq.getReadInstance(true)) { rsp: TouchControlResp
->
    // rsp.isRead(), rsp.isTouch(), rsp.getAppType(), rsp.getSleepTime(),
    rsp.isTouchSleep()
    }

// Read gesture
CommandHandle.getInstance()
    .executeReqCmd(TouchControlReq.getReadInstance(false)) { rsp:
TouchControlResp ->
    // rsp.getAppType(), rsp.getStrength()
    }

// Set normal touch: appType=9, strength=5, sleepTime=3 minutes
TouchControlReq.getWriteInstance(9, true, 5, 3)

// Set gesture: appType=9, strength=5
TouchControlReq.getWriteInstance(9, false, 5)

// Set RT11 touch: appType=3, state=1
TouchControlReq.getWriteTpSleepInstance(3, 1)
```

8.3 Device Settings

The device page covers:

- `DegreeSwitchReq`
- `BatterySavingReq`

```
CommandHandle.getInstance()
    .executeReqCmd(DegreeSwitchReq.getWriteInstance(true, true), null)

CommandHandle.getInstance()
    .executeReqCmd(BatterySavingReq.getWriteInstance(true), null)
```

Req parameter reference

Type	Construction / parameter	Type	Description
<code>DegreeSwitchReq</code>	<code>getReadInstance()</code> / <code>getWriteInstance(enable, celsius)</code>	<code>Boolean</code>	Reads or writes temperature display and Celsius/Fahrenheit selection.
<code>BatterySavingReq</code>	<code>getReadInstance()</code> / <code>getWriteInstance(enable)</code>	<code>Boolean</code>	Reads or writes battery-saving mode.

8.4 Camera Control

The device page includes two entries:

- Start camera control: `display_start_camera_control`
- Stop camera control: `display_stop_camera_control`

```
CommandHandle.getInstance()
    .executeReqCmd(CameraReq(CameraReq.ACTION_INTRO_CAMARA_UI), null)

BleOperateManager.getInstance()
    .addNotifyListener(
        Constants.CMD_TAKING_PICTURE.toInt(),
        ICommandResponse<CameraNotifyRsp> { rsp ->
            when (rsp.action) {
                CameraNotifyRsp.ACTION_TAKE_PHOTO -> {
                    // The device requests the app to take a photo.
                }
                CameraNotifyRsp.ACTION_FINISH -> {
                    // The device exits the camera page.
                }
            }
        }
    )

CommandHandle.getInstance()
    .executeReqCmd(CameraReq(CameraReq.ACTION_FINISH), null)
```

Req/Rsp parameter reference

Type	Construction / field	Type / range	Description
CameraReq	CameraReq(action)	Byte, ACTION_INT0_CAMARA_UI, ACTION_KEEP_SCREEN_ON, ACTION_FINISH	Opens the camera UI, keeps it awake, or closes it.
CameraNotifyRsp	action	Int, ACTION_INT0_CAMERA_UI, ACTION_TAKE_PHOTO, ACTION_FINISH	Device-side camera event; take a photo or close the page according to the action.

8.5 Wear Calibration

The device page can open `RevisionActivity`. Use wear calibration when heart rate cannot be measured, or when values are produced during empty detection.

```
BleOperateManager.getInstance().ringCalibration(true, listener)  
BleOperateManager.getInstance().ringCalibration(false, null)
```

9. Notification and Message Push

Sample files:

- `NotificationSettingActivity`
- `MyNotificationService`

9.1 Notification Switches

```
val req = SetANCSReq().apply {  
    setCall(true)  
    setSms(true)  
    setQq(true)  
    setWechat(true)  
}  
CommandHandle.getInstance().executeReqCmd(req) { rsp ->  
    // rsp.status == BaseRspCmd.RESULT_OK  
}  
  
CommandHandle.getInstance()  
    .executeReqCmd(  
        SimpleKeyReq(Constants.CMD_GET_ANCS_ON_OFF),  
        ICommandResponse<ReadANCSRsp> { rsp ->  
            // rsp.isCall, rsp.isSms, rsp.iswechat  
        }  
    )
```

Req/Rsp parameter reference

Type	Construction / field	Type	Description
<code>SetANCSReq</code>	<code>SetANCSReq()</code> and <code>setCall/setSms/setQq/setwechat/...</code>	<code>Boolean</code> per app	Enables/disables call, SMS, QQ, WeChat, Facebook, WhatsApp, Twitter, Skype, Line, LinkedIn, Instagram, TIM, Snapchat, and other notifications. <code>setAllOpen()</code> enables all supported bits.
<code>SetANCSReq</code>	<code>SetANCSReq(stateMask, ancsMask)</code>	two <code>Short</code> bitmasks	Low-level constructor; prefer named setters to avoid bit-position mistakes.
<code>ReadANCSRsp</code>	<code>stateMask, ancsMask</code>	<code>Short</code>	Raw switch bitmasks.
<code>ReadANCSRsp</code>	<code>call, sms, qq, wechat, facebook, whatsapp, twitter, skype, line, linkedin, instagram, tim, snapchat, other</code>	<code>Boolean</code>	Decoded notification switches.

9.2 Real Notification Forwarding

External apps need to declare a notification listener service and guide users to grant notification access.

```
<service
    android:name=".service.MyNotificationService"
    android:exported="false"
    android:label="@string/app_name"
    android:permission="android.permission.BIND_NOTIFICATION_LISTENER_SERVICE">
    <intent-filter>
        <action
            android:name="android.service.notification.NotificationListenerService" />
        </intent-filter>
    </service>
```

Map app package names to message types in the service, then call:

```
MessPushUtil.pushMsg(type, "your message")
```

Common types:

type	Meaning
<code>0x00</code>	Incoming call

type	Meaning
0x01	SMS
0x02	QQ
0x03	WeChat
0x05	Facebook
0x06	WhatsApp
0x07	Twitter
0x08	Skype
0x09	Line
0x0a	LinkedIn
0x0b	Instagram
0x0d	Snapchat
0x0e	Other notification

`PushMsgUintReq` is the low-level request behind `MessPushUtil.pushMsg(...)`:

Construction / parameter	Type / range	Description
<code>PushMsgUintReq(type, messageId, length, payload)</code>	<code>Byte</code> , <code>Int</code> , <code>Int</code> , <code>ByteArray</code>	Notification type, message/fragment identifier, payload length, and encoded message bytes. Prefer <code>MessPushUtil</code> so encoding and fragmentation remain consistent.

9.3 Notification Event Notify

Use `NotificationEventNotifyReq` to send device notification events, custom light notifications, and custom vibration notifications.

```

CommandHandle.getInstance()
    .executeReqCmd(NotificationEventNotifyReq.getEventInstance(eventType)) { }

CommandHandle.getInstance()
    .executeReqCmd(
        NotificationEventNotifyReq.getCustomLightInstance(
            priority,
            light,
            loopCount,
            activeDuration,
            pauseDuration
        )
    ) { }

```

```

CommandHandle.getInstance()
    .executeReqCmd(
        NotificationEventNotifyReq.getCustomVibrationInstance(
            priority,
            vibration,
            loopCount,
            activeDuration,
            pauseDuration
        )
    ) { }

```

Event type values:

eventType	Meaning
0	NULL
1	Double-click button event
2	Pairing event
3	"Miss you" event
4	"Love you" event
5	"Need you" event
6	Sad event
7	Love-letter event
8	Custom light notification
9	Custom vibration notification

Custom light values:

light	Meaning
0	NULL
1	Stop
2	Green light
3	Red light

Custom vibration values:

vibration	Meaning
0	NULL
1	Stop
2	Start

Common custom-notification parameter ranges:

Parameter	Range	Unit / Notes
<code>priority</code>	<code>0..255</code>	Single-byte protocol field. Larger values have higher priority. Sample presets: <code>0/1/3/5/10</code> .
<code>loopCount</code>	<code>0..255</code>	Single-byte protocol field. Sample presets: <code>1/3/5/10</code> .
<code>activeDuration</code>	<code>1..127</code>	Unit is <code>100ms</code> ; used as light-on duration or vibration duration. Sample presets: <code>1/5/10/20/50/127</code> .
<code>pauseDuration</code>	<code>0..255</code>	Unit is <code>100ms</code> ; single-byte protocol field. Sample presets: <code>0/5/10/20/50</code> .

`NotificationEventNotifyReq` factory parameters:

Factory	Parameters	Description
<code>getEventInstance(eventType)</code>	<code>eventType</code> <code>0..9</code>	Sends one of the event constants listed above.
<code>getCustomLightInstance(priority, light, loopCount, activeDuration, pauseDuration)</code>	ranges listed above	Sends a custom light pattern.
<code>getCustomVibrationInstance(priority, vibration, loopCount, activeDuration, pauseDuration)</code>	ranges listed above	Sends a custom vibration pattern.

9.4 DND and Music Switch

```
CommandHandle.getInstance()
    .executeReqCmd(DndReq.getInstance(true, StartEndTimeEntity(22, 0, 7, 0))) { }
```

```
CommandHandle.getInstance()
    .executeReqCmd(MusicSwitchReq.getInstance(true)) { }
```

Req parameter reference

Type	Construction / parameter	Type	Description
<code>DndReq</code>	<code>getInstance()</code> / <code>getInstance(enable, timeRange)</code>	<code>Boolean</code> , <code>StartEndTimeEntity</code>	Reads or writes DND and its start/end hour and minute. Overnight ranges such as <code>22:00..07:00</code> are supported by compatible firmware.
<code>MusicSwitchReq</code>	<code>getInstance()</code> / <code>getInstance(enable)</code>	<code>Boolean</code>	Reads or writes music control.

Type	Construction / parameter	Type	Description
MusicSwitchReq	getNewWriteInstance(enable, firstValue, secondValue, text)	Boolean, two Int, String	Extended firmware form; values/text are product-specific, so use it only with the matching capability and Sample mapping.

10. Advanced

Sample file: `AdvancedSettingActivity`.

10.1 Device Capability

Read and cache device capability after connection:

```
CommandHandle.getInstance()
    .executeReqCmd(
        DeviceSupportReq.getReadInstance(),
        ICommandResponse<DeviceSupportFunctionRsp> { rsp ->
            // supportTouch, supportGesture, supportRingMusic, supportMuslin,
etc.
        }
    )
```

The Sample caches capability in `SetTime` and `DeviceFunctionSupport`, and shows a not-supported prompt before executing unsupported features.

10.2 AGPS, Intel, Muslim/Praise

```
CommandHandle.getInstance().executeReqCmd(AgpsReq.getWriteInstance(true)) { }
CommandHandle.getInstance().executeReqCmd(IntelReq.getWriteInstance(true,
5.toByte())) { }
CommandHandle.getInstance().executeReqCmd(MuslimReq.getWriteInstance(true)) { }
CommandHandle.getInstance().executeReqCmd(MuslimTargetReq(33)) { }
```

RT11 Muslim/praise touch entry:

```
TouchControlReq.getWriteTpsleepInstance(3, 1)
```

Normal touch/gesture entry:

```
TouchControlReq.getWriteInstance(9, true, 5) // open touch
TouchControlReq.getWriteInstance(0, true, 5) // close touch
TouchControlReq.getWriteInstance(9, false, 5) // open gesture
TouchControlReq.getWriteInstance(0, false, 5) // close gesture
```

Req parameter reference

Type	Construction / parameter	Type / range	Description
<code>AgpsReq</code>	<code>getReadInstance()</code> / <code>getWriteInstance(enable)</code>	<code>Boolean</code>	Reads or writes AGPS enable state.
<code>IntellReq</code>	<code>getReadInstance()</code> / <code>getWriteInstance(enable, value)</code>	<code>Boolean</code> , <code>Byte</code>	Reads or writes the intelligent-feature switch and firmware-defined level/value; the Sample uses 5.
<code>MuslimReq</code>	<code>MuslimReq(dayOffset)</code>	<code>Byte</code> , retained day	Reads Muslim/praise history for the selected day.
<code>MuslimReq</code>	<code>getWriteInstance(enable)</code> / <code>MuslimReq(enable)</code>	<code>Boolean</code>	Enables or disables the Muslim/praise function.
<code>MuslimTargetReq</code>	<code>MuslimTargetReq(target[, enable])</code>	<code>Int</code> , optional <code>Boolean</code>	Writes the praise/count target and optional enable state; the Sample target is 33.

10.3 High-Risk Operations

The device page provides shutdown, reboot, and restore-factory entries with confirmation dialogs. The advanced page keeps close-device-Bluetooth and shutdown:

- Shut down device: `BleOperateManager.getInstance().shutdown()`
- Reboot device: `RestoreKeyReq(Constants.CMD_RE_BOOT)`
- Restore factory settings: `RestoreKeyReq(Constants.CMD_RE_STORE)`
- Close device Bluetooth: `BluetoothCloseReq.getWriteInstance()`

External apps should keep confirmation dialogs or a second-confirmation flow to avoid accidental disconnects, reboots, or data loss.

```

BleOperateManager.getInstance().shutdown()

CommandHandle.getInstance()
    .executeReqCmd(RestoreKeyReq(Constants.CMD_RE_BOOT), null)

CommandHandle.getInstance()
    .executeReqCmd(RestoreKeyReq(Constants.CMD_RE_STORE), null)

CommandHandle.getInstance()
    .executeReqCmd(BluetoothCloseReq.getwriteInstance(), null)

```

Req/Rsp parameter reference

Type	Construction / field	Type / range	Description
RestoreKeyReq	RestoreKeyReq(command)	Byte, Constants.CMD_RE_BOOT or CMD_RE_STORE	Reboots or restores factory settings. Always require user confirmation.
BluetoothCloseReq	getwriteInstance()	No business parameter	Turns off device Bluetooth.
BluetoothCloseRsp	no business field	Status inherited from BaseRspCmd	Check the common command status to determine success.
BluetoothEnableReq	getwriteInstance(enable)	Int, 0 off / 1 on	Legacy Bluetooth switch request. Closing Bluetooth disconnects the app.

11. OTA

Sample file: `OtaActivity`.

Typical flow:

1. Select a `.bin` firmware file.
2. Validate the file path.
3. Start the DFU handler.
4. Listen for progress, success, and failure.

The legacy task-chain example also demonstrates `dfu(...)` as a chain step. If firmware update and data sync run sequentially, use a task chain to manage timeouts and failures.

12. Device-Initiated Reports

Register device report listeners after connection:

```

BleOperateManager.getInstance()
    .addOutDeviceListener(ListenerKey.Heart, myDeviceNotifyListener)

BleOperateManager.getInstance()
    .removeNotifyListener(ListenerKey.Heart)

```

Use `DeviceNotifyRsp.TYPE_*` constants in callbacks instead of hard-coded `dataType` numbers:

```

override fun onDataResponse(resultEntity: DeviceNotifyRsp?) {
    if (resultEntity?.status != BaseRspCmd.RESULT_OK) return

    when (resultEntity.dataType) {
        DeviceNotifyRsp.TYPE_BATTERY -> {
            // loadData[1] is battery level, loadData[2] is charging state;
            // DeviceNotifyRsp.CHARGE_STATUS_CHARGING means charging.
        }
        DeviceNotifyRsp.TYPE_STEP_CALORIE_DISTANCE -> {
            // Real-time steps, calories, and distance.
        }
        DeviceNotifyRsp.TYPE_EMOTION -> {
            // Emotion state change; query emotion history with
            getEmotion/getEmotions.
        }
    }
}

```

Common `DeviceNotifyRsp` report types:

Constant	dataType	Meaning
<code>DeviceNotifyRsp.TYPE_HEART_RATE</code>	1	Heart-rate change
<code>DeviceNotifyRsp.TYPE_BLOOD_PRESSURE</code>	2	Blood-pressure change
<code>DeviceNotifyRsp.TYPE_BLOOD_OXYGEN</code>	3	Blood-oxygen change
<code>DeviceNotifyRsp.TYPE_STEP</code>	4	Step-detail change
<code>DeviceNotifyRsp.TYPE_BODY_TEMPERATURE</code>	5	Temperature change
<code>DeviceNotifyRsp.TYPE_SPORT_RECORD</code>	7	New sport record
<code>DeviceNotifyRsp.TYPE_BATTERY</code>	12 / 0x0c	Battery / charging state
<code>DeviceNotifyRsp.TYPE_STEP_CALORIE_DISTANCE</code>	18 / 0x12	Real-time steps, calories, and distance
<code>DeviceNotifyRsp.TYPE_EMOTION</code>	21 / 0x15	Emotion state change
<code>DeviceNotifyRsp.TYPE_MUSLIM_TASBIH_CLICK</code>	37 / 0x25	Muslim/praise real-time count

Constant	dataType	Meaning
<code>DeviceNotifyRsp.TYPE_CUSTOM_BUTTON_FUNCTION</code>	45 / 0x2d	Custom touch event
<code>DeviceNotifyRsp.TYPE_AUTO_HEART_RATE_ALARM</code>	58 / 0x3A	Heart-rate high/low alert
<code>DeviceNotifyRsp.TYPE_RECORD_FILE_COUNT</code>	209 / 0xD1	Device recording file count changed

`DeviceNotifyRsp` response fields:

Field	Type / range	Description
<code>dataType</code> / <code>unsignedDataType</code>	signed/unsigned <code>Int</code>	Report discriminator. Compare with <code>TYPE_*</code> constants, preferably <code>unsignedDataType</code> when handling values above <code>0x7f</code> .
<code>loadData</code>	<code>ByteArray</code>	Type-specific payload. Decode only according to the matching <code>TYPE_*</code> layout and SDK index/state constants.
inherited <code>status</code>	common response status	Ignore payloads whose status is not <code>BaseRspCmd.RESULT_OK</code> .

Remove listeners when they are no longer needed to avoid duplicate callbacks.

13. Legacy Supplemental Capabilities

The following capabilities exist in the legacy document but are not shown as main entries in the current Sample. Integrate them only when your product needs them.

13.1 App-Started Sport Mode

The app can control app-started sport mode with `PhoneSportReq.getSportStatus(status, sportType)`, where `sportType` should use an SDK constant from `SportTypeConstants`, and listen for real-time sport data with `BleOperateManager.addSportDeviceListener(0x78, listener)`. The current Sample entry is `SportRecordActivity`, which demonstrates sport sync, start sport, stop sport, and real-time sport notifications.

Start Sport

```
import com.oudmon.ble.base.communication.sport.SportTypeConstants

val sportType = SportTypeConstants.SPORT_TYPE_WALK // See the sportType table
below. It means walk.
CommandHandle.getInstance().executeReqCmd(
    PhoneSportReq.getSportStatus(1, sportType.toByte()),
    ICommandResponse<AppSportRsp> { rsp ->
        // rsp.gpsStatus: sport state returned by the device
        // rsp.timeStamp: sport timestamp when gpsStatus == 6
    }
)
```

Stop Sport

```
CommandHandle.getInstance().executeReqCmd(
    PhoneSportReq.getSportStatus(4, sportType.toByte()),
    ICommandResponse<AppSportRsp> { rsp ->
        // Check rsp.gpsStatus for the stop result.
    }
)
```

Products should check the sport duration before stopping. The Sample shows a confirmation dialog when the duration is less than one minute; confirming still sends the stop command so integrators can debug the flow.

Listen for Sport Data

```
private val listener = object : DeviceSportNotifyListener() {
    override fun onDataResponse(resultEntity: DeviceNotifyRsp?) {
        super.onDataResponse(resultEntity)
        if (resultEntity?.status != BaseRspCmd.RESULT_OK) return
        val data = resultEntity.loadData ?: return
        if (data.size < 14) return

        val status = readUnsignedBigEndian(data, 1, 1)
        val durationSeconds = readUnsignedBigEndian(data, 2, 2)
        val heartRate = readUnsignedBigEndian(data, 4, 1)
        val steps = readUnsignedBigEndian(data, 5, 3)
        val distanceMeters = readUnsignedBigEndian(data, 8, 3)
        val calorie = readUnsignedBigEndian(data, 11, 3)

        Log.i("Sport", "heart rate: $heartRate, distance: ${distanceMeters /
1000.0}km, steps: $steps")
    }
}

BleOperateManager.getInstance().addSportDeviceListener(0x78, listener)

// Release the listener when the page is destroyed or notifications are no longer
needed.
BleOperateManager.getInstance().removeSportDeviceListener(0x78)
```

```
private fun readUnsignedBigEndian(data: ByteArray, start: Int, length: Int): Int
{
    var value = 0
    for (index in start until start + length) {
        value = (value shl 8) or (data[index].toInt() and 0xFF)
    }
    return value
}
```

Status and Field Reference

Type / field	Description
<code>PhoneSportReq.getSportStatus(status, sportType)</code>	<code>status</code> is the sport state, and <code>sportType</code> is a sport type constant from <code>SportTypeConstants</code> .
<code>status = 1</code>	Start sport.
<code>status = 2</code>	Pause sport.
<code>status = 3</code>	Continue sport.
<code>status = 4</code>	End sport.
<code>status = 6</code>	Sport start timestamp.
<code>AppSportRsp.gpsStatus</code>	Sport state returned by the device.
<code>AppSportRsp.timeStamp</code>	Unix timestamp in seconds when <code>gpsStatus == 6</code> .
<code>DeviceNotifyRsp.loadData[1]</code>	Device sport notification state.
<code>DeviceNotifyRsp.loadData[2..3]</code>	Sport duration in seconds.
<code>DeviceNotifyRsp.loadData[4]</code>	Heart rate.
<code>DeviceNotifyRsp.loadData[5..7]</code>	Steps.
<code>DeviceNotifyRsp.loadData[8..10]</code>	Distance in meters. Divide by <code>1000</code> for kilometers.
<code>DeviceNotifyRsp.loadData[11..13]</code>	Calories. The Sample follows the legacy app display convention and divides by <code>1000</code> .

sportType Values

SDK Constant	Type Value	English Name
<code>SportTypeConstants.SPORT_TYPE_WALK</code>	<code>4</code>	Walk
<code>SportTypeConstants.SPORT_TYPE_ROPE_SKIPPING</code>	<code>5</code>	Rope skipping

SDK Constant	Type Value	English Name
<code>SportTypeConstants.SPORT_TYPE_SWIMMING</code>	6	Swimming
<code>SportTypeConstants.SPORT_TYPE_RUN</code>	7	Run
<code>SportTypeConstants.SPORT_TYPE_HIKING</code>	8	Hiking
<code>SportTypeConstants.SPORT_TYPE_CYCLING</code>	9	Cycling
<code>SportTypeConstants.SPORT_TYPE_OTHER_SPORTS</code>	10	Other sports
<code>SportTypeConstants.SPORT_TYPE_SWING</code>	11	Swing
<code>SportTypeConstants.SPORT_TYPE_MOUNTAIN_CLIMBING</code>	20	Mountain climbing
<code>SportTypeConstants.SPORT_TYPE_BADMINTON</code>	21	Badminton
<code>SportTypeConstants.SPORT_TYPE_YOGA</code>	22	Yoga
<code>SportTypeConstants.SPORT_TYPE_AEROBICS</code>	23	Aerobics
<code>SportTypeConstants.SPORT_TYPE_SPINNING_BIKE</code>	24	Spinning bike
<code>SportTypeConstants.SPORT_TYPE_KAYAKING</code>	25	Kayaking
<code>SportTypeConstants.SPORT_TYPE_ELLIPTICAL_MACHINE</code>	26	Elliptical machine
<code>SportTypeConstants.SPORT_TYPE_ROWING_MACHINE</code>	27	Rowing machine
<code>SportTypeConstants.SPORT_TYPE_PINGPONG</code>	28	PingPong
<code>SportTypeConstants.SPORT_TYPE_TENNIS</code>	29	Tennis
<code>SportTypeConstants.SPORT_TYPE_GOLF</code>	30	Golf
<code>SportTypeConstants.SPORT_TYPE_BASKETBALL</code>	31	Basketball
<code>SportTypeConstants.SPORT_TYPE_SOCCER</code>	32	Football
<code>SportTypeConstants.SPORT_TYPE_VOLLEYBALL</code>	33	Volleyball
<code>SportTypeConstants.SPORT_TYPE_ROCK_CLIMBING</code>	34	Rock climbing
<code>SportTypeConstants.SPORT_TYPE_DANCE</code>	35	Dance
<code>SportTypeConstants.SPORT_TYPE_ROLLER_SKATING</code>	36	Roller skating
<code>SportTypeConstants.SPORT_TYPE_TREADMILL</code>	40	Treadmill
<code>SportTypeConstants.SPORT_TYPE_INDOOR_WALKING</code>	41	Indoor walking
<code>SportTypeConstants.SPORT_TYPE_TRAIL_RUNNING</code>	42	Trail running
<code>SportTypeConstants.SPORT_TYPE_WALKING</code>	43	Walking
<code>SportTypeConstants.SPORT_TYPE_PLAYGROUND_RUNNING</code>	44	Playground running
<code>SportTypeConstants.SPORT_TYPE_RUNNING_FOR_FAT_LOSS</code>	45	Running for fat loss
<code>SportTypeConstants.SPORT_TYPE_OUTDOOR_CYCLING</code>	50	Outdoor cycling
<code>SportTypeConstants.SPORT_TYPE_INDOOR_CYCLING</code>	51	Indoor cycling
<code>SportTypeConstants.SPORT_TYPE_MOUNTAIN_BIKING</code>	52	Mountain biking

SDK Constant	Type Value	English Name
<code>SportTypeConstants.SPORT_TYPE_BMX</code>	53	BMX
<code>SportTypeConstants.SPORT_TYPE_SWIMMING_IN_THE_POOL</code>	55	Swimming in the pool
<code>SportTypeConstants.SPORT_TYPE_OUTDOOR_SWIMMING</code>	56	Outdoor swimming
<code>SportTypeConstants.SPORT_TYPE_FIN_SWIMMING</code>	57	Fin swimming
<code>SportTypeConstants.SPORT_TYPE_SYNCHRONIZED_SWIMMING</code>	58	Synchronized swimming
<code>SportTypeConstants.SPORT_TYPE_OUTDOOR_HIKING</code>	60	Outdoor hiking
<code>SportTypeConstants.SPORT_TYPE_ORIENTEERING</code>	61	Orienteering
<code>SportTypeConstants.SPORT_TYPE_FISHING</code>	62	Fishing
<code>SportTypeConstants.SPORT_TYPE_HUNTING</code>	63	Hunting
<code>SportTypeConstants.SPORT_TYPE_SKATEBOARD</code>	64	Skateboard
<code>SportTypeConstants.SPORT_TYPE_PARKOUR</code>	65	Parkour
<code>SportTypeConstants.SPORT_TYPE_ATV</code>	66	ATV
<code>SportTypeConstants.SPORT_TYPE_MOTOCROSS</code>	67	Motocross
<code>SportTypeConstants.SPORT_TYPE_RACING</code>	68	Racing
<code>SportTypeConstants.SPORT_TYPE_HAND_CRANK</code>	69	Hand crank
<code>SportTypeConstants.SPORT_TYPE_MARATHON</code>	70	Marathon
<code>SportTypeConstants.SPORT_TYPE_HURDLE_RACE</code>	71	Hurdle Race
<code>SportTypeConstants.SPORT_TYPE_STAIR_CLIMBER</code>	80	Stair Climber
<code>SportTypeConstants.SPORT_TYPE_STEPPER</code>	81	Stepper
<code>SportTypeConstants.SPORT_TYPE_MIXED_AEROBIC</code>	82	Mixed Aerobic
<code>SportTypeConstants.SPORT_TYPE_KICKBOXING</code>	83	Kickboxing
<code>SportTypeConstants.SPORT_TYPE_CORE_TRAINING</code>	84	Core training
<code>SportTypeConstants.SPORT_TYPE_CROSS_TRAINING</code>	85	Cross-training
<code>SportTypeConstants.SPORT_TYPE_INDOOR_FITNESS</code>	86	Indoor fitness
<code>SportTypeConstants.SPORT_TYPE_GROUP_GYMNASTICS</code>	87	Group Gymnastics
<code>SportTypeConstants.SPORT_TYPE_STRENGTH_TRAINING</code>	88	Strength training
<code>SportTypeConstants.SPORT_TYPE_INTERVAL_TRAINING</code>	89	Interval training
<code>SportTypeConstants.SPORT_TYPE_FREE_TRAINING</code>	90	Free training
<code>SportTypeConstants.SPORT_TYPE_FLEXIBILITY_TRAINING</code>	91	Flexibility training
<code>SportTypeConstants.SPORT_TYPE_GYMNASTICS</code>	92	Gymnastics
<code>SportTypeConstants.SPORT_TYPE_STRETCH</code>	93	Stretch

SDK Constant	Type Value	English Name
SportTypeConstants.SPORT_TYPE_PILATES	94	Pilates
SportTypeConstants.SPORT_TYPE_HORIZONTAL_BAR	95	Horizontal bar
SportTypeConstants.SPORT_TYPE_DOUBLE_BARS	96	Double bars
SportTypeConstants.SPORT_TYPE_BATTLE_ROPE	97	Battle Rope
SportTypeConstants.SPORT_TYPE_FITNESS_EXERCISE	98	Fitness exercise
SportTypeConstants.SPORT_TYPE_BALANCE_TRAINING	99	Balance training
SportTypeConstants.SPORT_TYPE_STEP_TRAINING	100	Step training
SportTypeConstants.SPORT_TYPE_SQUARE_DANCE	110	Square dance
SportTypeConstants.SPORT_TYPE_BALLROOM_DANCE	111	Ballroom Dance
SportTypeConstants.SPORT_TYPE_BELLY_DANCE	112	Belly dance
SportTypeConstants.SPORT_TYPE_BALLET	113	Ballet
SportTypeConstants.SPORT_TYPE_HIP_HOP	114	Hip-hop
SportTypeConstants.SPORT_TYPE_ZUMBA	115	Zumba
SportTypeConstants.SPORT_TYPE_LATIN_DANCE	116	Latin Dance
SportTypeConstants.SPORT_TYPE_JAZZ_DANCE	117	Jazz Dance
SportTypeConstants.SPORT_TYPE_HIP_HOP_DANCE	118	Hip-hop dance
SportTypeConstants.SPORT_TYPE_POLE_DANCE	119	pole dance
SportTypeConstants.SPORT_TYPE_BREAK_DANCE	120	Break dance
SportTypeConstants.SPORT_TYPE_FOLK_DANCE	121	Folk Dance
SportTypeConstants.SPORT_TYPE_MUSIC_DANCE	122	Music Dance
SportTypeConstants.SPORT_TYPE_MODERN_DANCE	123	Modern Dance
SportTypeConstants.SPORT_TYPE_DISCO	124	Disco
SportTypeConstants.SPORT_TYPE_TAP_DANCE	125	Tap dance
SportTypeConstants.SPORT_TYPE_OTHER_DANCES	126	Other dances
SportTypeConstants.SPORT_TYPE_BOXING	130	Boxing
SportTypeConstants.SPORT_TYPE_WRESTLING	131	Wrestling
SportTypeConstants.SPORT_TYPE_MARTIAL_ARTS	132	Martial Arts
SportTypeConstants.SPORT_TYPE_TAI_CHI	133	Tai Chi
SportTypeConstants.SPORT_TYPE_MUAY_THAI	134	Muay Thai
SportTypeConstants.SPORT_TYPE_JUDO	135	Judo
SportTypeConstants.SPORT_TYPE_TAEKWONDO	136	Taekwondo
SportTypeConstants.SPORT_TYPE_KARATE	137	Karate

SDK Constant	Type Value	English Name
<code>SportTypeConstants.SPORT_TYPE_FREE_FIGHT</code>	138	Free fight
<code>SportTypeConstants.SPORT_TYPE_SWORDSMANSHIP</code>	139	Swordsmanship
<code>SportTypeConstants.SPORT_TYPE_JIU_JITSU</code>	140	jiu-jitsu
<code>SportTypeConstants.SPORT_TYPE_FENCING_141</code>	141	Kendo
<code>SportTypeConstants.SPORT_TYPE_FENCING_142</code>	142	Kendo
<code>SportTypeConstants.SPORT_TYPE_BEACH_FOOTBALL</code>	150	Beach football
<code>SportTypeConstants.SPORT_TYPE_BEACH_VOLLEYBALL</code>	151	Beach Volleyball
<code>SportTypeConstants.SPORT_TYPE_BASEBALL</code>	152	Baseball
<code>SportTypeConstants.SPORT_TYPE_SOFTBALL</code>	153	Softball
<code>SportTypeConstants.SPORT_TYPE_RUGBY</code>	154	Football
<code>SportTypeConstants.SPORT_TYPE_HOCKEY</code>	155	Hockey
<code>SportTypeConstants.SPORT_TYPE_SQUASH</code>	156	Squash
<code>SportTypeConstants.SPORT_TYPE_GATEBALL</code>	157	Gateball
<code>SportTypeConstants.SPORT_TYPE_CRICKET</code>	158	Cricket
<code>SportTypeConstants.SPORT_TYPE_HANDBALL</code>	159	Handball
<code>SportTypeConstants.SPORT_TYPE_BOWLING</code>	160	Bowling
<code>SportTypeConstants.SPORT_TYPE_POLO</code>	161	Polo
<code>SportTypeConstants.SPORT_TYPE_RACQUETBALL</code>	162	Squash
<code>SportTypeConstants.SPORT_TYPE_BILLIARDS</code>	163	Billiards
<code>SportTypeConstants.SPORT_TYPE_TAKRAW_BALL</code>	164	Takraw ball
<code>SportTypeConstants.SPORT_TYPE_DODGEBALL</code>	165	Dodgeball
<code>SportTypeConstants.SPORT_TYPE_WATER_POLO</code>	166	Water polo
<code>SportTypeConstants.SPORT_TYPE_PUCK</code>	167	Puck
<code>SportTypeConstants.SPORT_TYPE_SHUTTLECOCK</code>	168	Shuttlecock
<code>SportTypeConstants.SPORT_TYPE_INDOOR_FOOTBALL</code>	169	Indoor football
<code>SportTypeConstants.SPORT_TYPE_SANDBAG</code>	170	Sandbag
<code>SportTypeConstants.SPORT_TYPE_BOCCE</code>	171	Bocce
<code>SportTypeConstants.SPORT_TYPE_PALACE_BALL</code>	172	Palace Ball
<code>SportTypeConstants.SPORT_TYPE_FLOOR_BALL</code>	173	floor ball
<code>SportTypeConstants.SPORT_TYPE_AUSTRALIAN_RULES_FOOTBALL</code>	174	Australian Rules Football
<code>SportTypeConstants.SPORT_TYPE_PICKBALL</code>	175	Pickball
<code>SportTypeConstants.SPORT_TYPE_OUTDOOR_BOATING</code>	180	Outdoor boating

SDK Constant	Type Value	English Name
SportTypeConstants.SPORT_TYPE_SAILING	181	Sailing
SportTypeConstants.SPORT_TYPE_DRAGON_BOAT	182	Dragon Boat
SportTypeConstants.SPORT_TYPE_SURFING	183	surfing
SportTypeConstants.SPORT_TYPE_KITESURFING	184	Kitesurfing
SportTypeConstants.SPORT_TYPE_PADDLE	185	Paddle
SportTypeConstants.SPORT_TYPE_PADDLEBOARD	186	Paddleboard
SportTypeConstants.SPORT_TYPE_INDOOR_SURFING	187	Indoor surfing
SportTypeConstants.SPORT_TYPE_RAFTING	188	rafting
SportTypeConstants.SPORT_TYPE_SNORKELING	189	Snorkeling
SportTypeConstants.SPORT_TYPE_SKIING	190	Snowboard
SportTypeConstants.SPORT_TYPE_SNOWBOARDING	191	Snowboard
SportTypeConstants.SPORT_TYPE_ALPINE_SKIING	192	Alpine skiing
SportTypeConstants.SPORT_TYPE_CROSS_COUNTRY_SKIING	193	Cross-country skiing
SportTypeConstants.SPORT_TYPE_SKI_ORIENTEERING	194	Orienteering
SportTypeConstants.SPORT_TYPE_BATHATHLON	195	Bathathlon
SportTypeConstants.SPORT_TYPE_OUTDOOR_SKATING	196	Outdoor Skating
SportTypeConstants.SPORT_TYPE_INDOOR_SKATING	197	Indoor skating
SportTypeConstants.SPORT_TYPE_CURLING	198	Curling
SportTypeConstants.SPORT_TYPE_BOBSLEIGH	199	Snowmobile
SportTypeConstants.SPORT_TYPE_SLEIGH	200	Sleigh
SportTypeConstants.SPORT_TYPE_SNOWMOBILE	201	Snowmobile
SportTypeConstants.SPORT_TYPE_SNOWSHOEING	202	Snowshoeing
SportTypeConstants.SPORT_TYPE_HULA_HOOP	210	Hula hoop
SportTypeConstants.SPORT_TYPE_FRISBEE	211	frisbee
SportTypeConstants.SPORT_TYPE_DARTS	212	Darts
SportTypeConstants.SPORT_TYPE_KITE_FLYING	213	Kite flying
SportTypeConstants.SPORT_TYPE_TUG_OF_WAR	214	tug of war
SportTypeConstants.SPORT_TYPE_ESPORTS	215	Esports
SportTypeConstants.SPORT_TYPE_WANDERER	216	Wanderer
SportTypeConstants.SPORT_TYPE_SWING_SET	217	Swing
SportTypeConstants.SPORT_TYPE_SHUFFLEBOARD	218	Shuffleboard
SportTypeConstants.SPORT_TYPE_TABLE_FOOTBALL	219	Football

SDK Constant	Type Value	English Name
<code>SportTypeConstants.SPORT_TYPE_SOMATOSENSORY_GAME</code>	220	Somatosensory game
<code>SportTypeConstants.SPORT_TYPE_BUNGEE_JUMPING</code>	221	Bungee Jumping
<code>SportTypeConstants.SPORT_TYPE_PARACHUTING</code>	222	Parachuting
<code>SportTypeConstants.SPORT_TYPE_ANUSARA</code>	223	Anusara
<code>SportTypeConstants.SPORT_TYPE_YIN_YOGA</code>	224	Yin Yoga
<code>SportTypeConstants.SPORT_TYPE_PREGNANCY_YOGA</code>	225	Pregnancy Yoga
<code>SportTypeConstants.SPORT_TYPE_CHESS</code>	230	Chess
<code>SportTypeConstants.SPORT_TYPE_GO</code>	231	Go
<code>SportTypeConstants.SPORT_TYPE_CHECKERS</code>	232	Checkers
<code>SportTypeConstants.SPORT_TYPE_BOARD_GAMES</code>	233	Board Games
<code>SportTypeConstants.SPORT_TYPE_BRIDGE</code>	234	Bridge
<code>SportTypeConstants.SPORT_TYPE_TRIATHLON</code>	235	Triathlon
<code>SportTypeConstants.SPORT_TYPE_ARCHERY</code>	236	Archery
<code>SportTypeConstants.SPORT_TYPE_COMPOUND_EXERCISE</code>	237	Compound exercise
<code>SportTypeConstants.SPORT_TYPE_DRIVING</code>	238	Driving

13.2 Muslim Data Sync

```
CommandHandle.getInstance()
    .executeReqCmd(MuslimReq(0), ICommandResponse<MuslimRsp> {
        // pressureArray: one data point per hour.
    })
```

For real-time count, see device-initiated report `DeviceNotifyRsp.TYPE_MUSLIM_TASBIH_CLICK`.

`MuslimRsp` response fields:

Field	Type / range	Description
<code>offset</code> , <code>endFlag</code> , <code>range</code> , <code>today</code>	<code>Int</code> , <code>Boolean</code> , <code>Int</code> , date	Day offset, transfer completion, sample range, and record date.
<code>pressureArray</code>	<code>ByteArray</code>	Legacy name for the Muslim/praise count data returned for the selected day.

13.3 Contacts, Watch Faces, and File Transfer

Legacy `Test.kt` keeps examples for contact sync, watch-face download, and file transfer. External projects should turn those test snippets into dedicated business flows:

- Contacts: `LargeDataHandler.getInstance().syncContactMore(list) { }`

- Watch faces: download the watch-face file, then send it through `FileHandle`.
- File transfer: register `SimpleCallback`, call `FileHandle.initRegister()`, and release callbacks after completion.
- Resource name query: register a dedicated `SimpleCallback`, call `FileHandle.getInstance().initRegister()`, then call `FileHandle.getInstance().start()`. Read the device resource file names from `onFileNames(ArrayList<String>)`. Remove this callback after a result or error, and write the names to the app log or debug log panel so the device-side resource state is visible during integration.
- Resource file update: let the user select a resource file and show its path, then call `FileHandle.getInstance().executeFilePrepare(path)` to prepare the bytes. After prepare succeeds, register the update callback, call `FileHandle.getInstance().initRegister()`, and send `FileHandle.getInstance().cmdFileInit(File(path).name)`. Show transfer progress from `onProgress(percent)`, and clean up the callback from `onComplete()` or the failing `onActionResult(type, errCode)` branch. Also unregister callbacks and call `endAndRelease()` when the page is destroyed.

13.4 One-Click Measurement

Legacy test entries include one-click measurement examples:

```
CommandHandle.getInstance()
    .executeReqCmd(

    StartHeartRateReq.getSimpleReq(StartHeartRateReq.TYPE_One_Click_Measurement),
        ICommandResponse<StopHeartRateRsp> { rsp ->
            // heart, hrv, bloodOxygen, stress, temperature
        }
    )
```

Raw PPG is now documented under 6.3 Health Settings. Check device capability before integrating this flow, and send the matching stop command when the measurement ends.

13.5 Device Recording PCM Stream Integration

Recording import APIs are provided by the SDK package `com.oudmon.ble.base.recording`. The SDK-facing contract is now “import device recordings as PCM streams”: host applications create `RecordingPcmImportManager` and receive the device recording list, PCM format, PCM byte chunks, progress, completion, and errors. Whether the device source is Opus, whether the transfer uses BLE legacy / BLE V2 / Wi-Fi, and Wi-Fi connection/retry details all stay inside the SDK.

```
val support = DeviceSupportFunctionRsp().apply {
    supportAudio = cachedSupport.supportAudio
    supportWifiImport = cachedSupport.supportWifiImport
    supportAudioCoding = cachedSupport.supportAudioCoding
    isOpusAudio = cachedSupport.isOpusAudio
}

val manager = RecordingPcmImportManager(
    context = activity,
```

```

        deviceAddress = deviceAddress,
        deviceName = deviceName,
        support = support,
        listener = object : RecordingPcmImportListener {
            override fun onRecordingsChanged(recordings:
List<RecordingDeviceRecording>) {
                // Display device recordings. Local-file matching belongs to the host
app.
            }

            override fun onImportStarted(recording: RecordingDeviceRecording, format:
RecordingPcmFormat) {
                // Create the host app's .pcm output stream, or a temporary PCM file
for WAV conversion.
            }

            override fun onPcmData(recording: RecordingDeviceRecording, data:
ByteArray) {
                // Write PCM bytes to the host app's FileOutputStream or data
pipeline.
            }

            override fun onImportProgress(recording: RecordingDeviceRecording,
percent: Int) {
                // Update import progress.
            }

            override fun onImportCompleted(recording: RecordingDeviceRecording) {
                // Close the output stream; convert PCM to WAV in the host app if
needed.
            }

            override fun onError(recording: RecordingDeviceRecording?, message:
String, cause: Throwable?) {
                // Clean up temporary files or output streams created by the host
app.
            }
        }
    }
}
)

```

The Sample `RecordingActivity` demonstrates host-side file ownership. The code constant `SAVE_RECORDING_AS_WAV` chooses the final file format. When it is `false`, `onImportStarted()` creates a `.pcm` file, `onPcmData()` writes incoming bytes, and `onImportCompleted()` closes and registers the local file. When it is `true`, the Sample first writes a temporary PCM file and then uses the Sample-local `wavFileWriter` to generate a WAV file. The SDK does not return local file paths, does not provide a `FileProvider`, does not delete host local files, and does not expose a WAV writer; those behaviors belong to the host app.

Device recording count changes are still reported through `DeviceNotifyRsp.TYPE_RECORD_FILE_COUNT`. The Sample calls `RecordingFileCountNotifyParser.parse(response)` and treats the result as a change signal, then refreshes the device list through `manager.loadRecordings()`. If an operation is busy, store `pendingRecordingFileCountRefresh` and refresh once after `onBusyChanged(false)`.

`RecordingDeviceRecording.sourceSizeBytes` is the device-side source/transfer size.

`RecordingPcmFormat` describes the final PCM stream: sample rate, channels, and bit depth. On Android 13+, if the device supports Wi-Fi import, the host must request `NEARBY_WIFI_DEVICES` before importing; Android 10–12 requires fine location permission. This feature only imports recordings already present on the device: it does not require `RECORD_AUDIO` and has no meeting-minutes, language-selection, or speech-to-text dependency.

14. Integration Recommendations

- After `onServiceDiscovered()`, request `SetTimeReq` and `DeviceSupportReq.getReadInstance()` first. After `SetTimeRsp` / `DeviceSupportFunctionRsp` is returned and support capabilities are cached, then sync or query health data.
- Before executing each feature entry, check that system Bluetooth is enabled and the device is connected. If Bluetooth is off, prompt the user to turn it on instead of sending commands.
- Keep second confirmation for restore-factory, close-Bluetooth, shutdown, and reboot operations.
- Separate "notification switch settings" from "real notification forwarding". ANCS controls switches; `MessPushUtil.pushMsg` sends the actual message content.
- For large-data sync, file transfer, DFU, and task chains, implement timeout, failure, and cleanup logic.